1. Explain the time and space complexity of your algorithm by showing and summing up the complexity of each subsection of your code.

a. [10 points] Your analysis should show that your unrestricted algorithm is at most O(nm) time and space.

The unrestricted algorithm contains a double for() loop with the first layer stopping at the size of sequence #1 (n) and the second layer stopping at the size of sequence #2 (m), this makes the time complexity of the algorithm O(nm). There are also two other for() loops with time complexity O(n) and O(m) and one while() loop with time complexity O(n+m) (they have no meaningful impact on the time complexity of the algorithm).

The unrestricted algorithm contains two 2D arrays where dimension #1 is the size of sequence #1 (n) and dimension #2 is the size of sequence #2 (m), this makes the space complexity of the algorithm O(nm).

b. [10 points] Your analysis should show that your banded algorithm is at most O(kn) time and space.

The banded algorithm contains a double for() loop with the first layer stopping at the size of the smaller sequence (n) and the second layer stopping at the size of the bandwidth (k), this makes the time complexity of the algorithm O(kn). There is also one other for() loop with time complexity O(k) and one while() loop with time complexity O(n+m) (they have no meaningful impact on the time complexity of the algorithm).

The banded algorithm contains a 2D array where dimension #1 is the size of the smaller sequence (n) and dimension #2 is the size of the bandwidth (k), this makes the space complexity of the algorithm O(kn).

2. [10 points] Write a paragraph that explains how your alignment extraction algorithm works, including the backtrace.

Create a cost 2D array and a backtrace 2D array. Fill out the first row (and column) of both. Visit each element in both arrays (left to right, top to bottom) and assign them a cost and backstep instruction based on which previous element (left, up, or diagonal) would give them the smallest possible value. Once finished, start at the very end of both 2D arrays (the bottom right corner). Continually jump to a previous element based on the backstep instruction of the current element until you have reached the start of both 2D arrays (the top left corner). While doing this add the cost of each step to the score and add the letters associated with that step to each new alignment string. Return the score and the new alignment strings (reversed to correct reversal).

3. [20 points] Include a “results” section showing both a screen-shot of your 10x10 score matrix for the unrestricted algorithm with align length n = 1000 and a screen-shot of your 10x10 score matrix for the banded algorithm with align length n = 3000.

Gene Sequence Alignment

	sequence1	sequence2	sequence3	sequence4	sequence5	sequence6	sequence7	sequence8	sequence9	sequence10
sequence1	-30	-1	4956	4956	4956	4956	4956	4956	4956	4956
sequence2		-33	4948	4948	4948	4948	4948	4948	4948	4948
sequence3			-3000	-2996	-2956	-2944	-1431	-1448	-1399	-1448
sequence4				-3000	-2960	-2948	-1431	-1448	-1399	-1448
sequence5					-3000	-2988	-1423	-1452	-1391	-1448
sequence6						-3000	-1426	-1452	-1394	-1448
sequence7							-3000	-2771	-2814	-2767
sequence8								-3000	-2731	-2996
sequence9									-3000	-2727
sequence10										-3000

Label I:

Sequence I:

Sequence J:

Label J:

Process Clear

☐ Banded Align Length: 1000

Done. Time taken: 18.895 seconds.

Gene Sequence Alignment

	sequence1	sequence2	sequence3	sequence4	sequence5	sequence6	sequence7	sequence8	sequence9	sequence10
sequence1	-30	-1	inf	inf	inf	inf	inf	inf	inf	inf
sequence2		-33	inf	inf	inf	inf	inf	inf	inf	inf
sequence3			-9000	-8984	-8888	-8848	-2735	-2743	-1429	-2735
sequence4				-9000	-8888	-8848	-2739	-2748	-1426	-2740
sequence5					-9000	-8960	-2711	-2739	-1426	-2727
sequence6						-9000	-2708	-2728	-1415	-2716
sequence7							-9000	-8103	-1256	-8099
sequence8								-9000	-1310	-8980
sequence9									-9000	-1315
sequence10										-9000

Label I:

Sequence I:

Sequence J:

Label J:

Process Clear

☑ Banded Align Length: 3000

Done. Time taken: 0.747 seconds.

4. [10 points] Include in the “results” section the extracted alignment for the first 100 characters of sequences #3 and #10 (counting from 1), computed using the unrestricted algorithm with n = 1000. Display the sequences in a side-by-side fashion in such a way that matches, substitutions, and insertions/deletions are clearly discernible as shown above in the To Do section. Also include the extracted alignment for the same pair of sequences when computed using the banded algorithm and n = 3000.

```
gattgcgagcgatttgcgtgcgtgcatcccgcttc-actg--at-ctcttgttagatcttttcataatctaaactttataaaaacatccactccctgta-
-ataa-gagtgattggcgtccgtacgtacccttttctactctcaaactcttgttagtttaaatc-taatctaaactttataaa--cggc-acttcctgtgt

gattgcgagcgatttgcgtgcgtgcatcccgcttc-actg--at-ctcttgttagatcttttcataatctaaactttataaaaacatccactccctgta-
-ataa-gagtgattggcgtccgtacgtacccttttctactctcaaactcttgttagtttaaatc-taatctaaactttataaa--cggc-acttcctgtgt
```

5. [30 points] Attach your commented source code for both your unrestricted and banded algorithms.

```
#!/usr/bin/python3

from which_pyqt import PYQT_VER
if PYQT_VER == 'PYQT5':
	from PyQt5.QtCore import QLineF, QPointF
elif PYQT_VER == 'PYQT4':
	from PyQt4.QtCore import QLineF, QPointF
else:
	raise Exception('Unsupported Version of PyQt: {}'.format(PYQT_VER))

import math
import time
import random

# Used to compute the bandwidth for banded version
MAXINDELS = 3

# Used to implement Needleman-Wunsch scoring
MATCH = -3
INDEL = 5
SUB = 1

class GeneSequencing:

	def __init__( self ):
		self.banded = None

# This is the method called by the GUI.  _seq1_ and _seq2_ are two sequences to be aligned,
# _banded_ is a boolean that tells you whether you should compute a banded alignment or full alignment, and
# _align_length_ tells you how many base pairs to use in computing the alignment

	def align( self, seq1, seq2, banded, align_length):
		# Pre-algorithm setup.

		self.banded = banded
		self.MaxCharactersToAlign = align_length

		seq1 = ' ' + seq1
		seq2 = ' ' + seq2

		if (banded == False): #Unrestricted alignment algorithm: Time and space complexity of O(nm).
			#Cost and backtrace 2D array setup.

			width = min(align_length + 1, len(seq1))
			height = min(align_length + 1, len(seq2))

			E = [[float('inf') for j in range(width)] for i in range(height)]
			P = [[None for j in range(width)] for i in range(height)]
```

```
        #Alignment #1+#2 creation.

        E[0][0] = 0
        P[0][0] = 'b'
        for i in range (1, width):
                E[0][i] = E[0][i-1] + INDEL
                P[0][i] = 'l'
        for j in range(1, height):
                E[j][0] = E[j - 1][0] + INDEL
                P[j][0] = 'u'

        for j in range (1, height):
                for i in range (1, width):
                        l = E[j][i-1] + INDEL
                        mini = l
                        ins = 'l'

                        u = E[j-1][i] + INDEL
                        if (u < mini):
                                mini = u
                                ins = 'u'

                        if (seq1[i] != seq2[j]):
                                d = E[j - 1][i - 1] + SUB
                        else:
                                d = E[j - 1][i - 1] + MATCH
                        if (d < mini):
                                mini = d
                                ins = 'd'

                        E[j][i] = mini
                        P[j][i] = ins

        #Score and alignment #1+#2 extraction.

        cost = 0
        i = min(width-1, align_length)
        j = min(height-1, align_length)
        newseq1 = ""
        newseq2 = ""

        while (i > 0 or j > 0):
                if (i > 0 and P[j][i] == 'l'):
                        cost += 5
                        newseq1 = newseq1 + seq1[i]
                        newseq2 = newseq2 + '-'
                        i -= 1
                elif (j > 0 and P[j][i] == 'u'):
                        cost += 5
                        newseq1 = newseq1 + '-'
                        newseq2 = newseq2 + seq2[j]
                        j -= 1
                elif (i > 0 and j > 0 and P[j][i] == 'd'):
                        if (seq1[i] == seq2[j]):
                                cost -= 3
                        else:
                                cost += 1
                        newseq1 = newseq1 + seq1[i]
                        newseq2 = newseq2 + seq2[j]
                        i -= 1
                        j -= 1

        score = cost
        alignment1 = newseq1[::-1].format(width, align_length, ',BANDED' if banded else '')
        alignment2 = newseq2[::-1].format(height, align_length, ',BANDED' if banded else '')

else: #Banded alignment algorithm: Time and space complexity of O(kn).
        #Cost and backtrace 2D array setup.

        if (len(seq1) > len(seq2)):
```

```
        width = min(align_length + 1, len(seq1))
        height = min(align_length + 1, len(seq2))
        x = True
else:
        height = min(align_length + 1, len(seq1))
        width = min(align_length + 1, len(seq2))
        x = False

if (abs(width - height) > 100):
        score = float('inf')
        alignment1 = "No Alignment Possible"
        alignment2 = "No Alignment Possible"
        return {'align_cost': score, 'seqi_first100': alignment1, 'seqj_first100': alignment2}

wid = MAXINDELS*2 + 1
E = [[float('inf') for i in range(wid)] for j in range(height)]
P = [[None for i in range(wid)] for j in range(height)]

#Alignment #1+#2 creation.

E[0][3] = 0
P[0][3] = 'b'
for i in range(4, 7):
        E[0][i] = E[0][i - 1] + INDEL
        P[0][i] = 'l'

for j in range(1, height):
        for i in range(j-MAXINDELS, j+MAXINDELS+1):
                if (i >= 0 and i < width):
                        k = i + MAXINDELS - j

                        mini = float('inf')

                        if (k > 0):
                                l = E[j][k - 1] + INDEL
                                mini = l
                                ins = 'l'

                        if (k < 6):
                                u = E[j - 1][k + 1] + INDEL
                                if (u < mini):
                                        mini = u
                                        ins = 'u'

                        if (x):
                                a = i
                                b = j
                        else:
                                a = j
                                b = i

                        if (seq1[a] != seq2[b]):
                                d = E[j - 1][k] + SUB
                        else:
                                d = E[j - 1][k] + MATCH
                        if (d < mini):
                                mini = d
                                ins = 'd'

                        E[j][k] = mini
                        P[j][k] = ins

#Score and alignment #1+#2 extraction.

cost = 0
i = min(width - 1, align_length)
j = min(height - 1, align_length)
newseq1 = ""
newseq2 = ""
```

```
        while (i > 0 or j > 0):
                k = i + MAXINDELS - j

                if (x):
                        a = i
                        b = j
                else:
                        a = j
                        b = i

                if (i > 0 and P[j][k] == 'l'):
                        cost += 5
                        if (x):
                                newseq1 = newseq1 + seq1[a]
                                newseq2 = newseq2 + '-'
                        else:
                                newseq1 = newseq1 + '-'
                                newseq2 = newseq2 + seq2[b]
                        i -= 1
                elif (j > 0 and P[j][k] == 'u'):
                        cost += 5
                        if (x):
                                newseq1 = newseq1 + '-'
                                newseq2 = newseq2 + seq2[b]
                        else:
                                newseq1 = newseq1 + seq1[a]
                                newseq2 = newseq2 + '-'
                        j -= 1
                elif (i > 0 and j > 0 and P[j][k] == 'd'):
                        if (seq1[a] == seq2[b]):
                                cost -= 3
                        else:
                                cost += 1
                        newseq1 = newseq1 + seq1[a]
                        newseq2 = newseq2 + seq2[b]
                        i -= 1
                        j -= 1

        score = cost
        alignment1 = newseq1[::-1].format(width, align_length, ',BANDED' if banded else '')
        alignment2 = newseq2[::-1].format(height, align_length, ',BANDED' if banded else '')

return {'align_cost':score, 'seqi_first100':alignment1, 'seqj_first100':alignment2}
```